

Lab Report No 9
Understanding of Best First Search and Beam
Search algorithms
Artificial Intelligence



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

20th May, 2025

Department of Computer Engineering,
HITEC University, Taxila

Example Task No 1:

Solution:

Brief description (3-5 lines)

Best First Search Algorithm:

The Best First Search algorithm is a general approach to searching in a graph or tree. It is an uninformed search algorithm, meaning it doesn't use any information about the goal or the structure of the search space beyond the provided graph or tree.
--

The code

<pre># Define a simple graph (adjacency list) graph = { 'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F', 'G'], 'D': [], 'E': [], 'F': [], 'G': [] } # Define heuristic values for each node (lower means closer to goal) heuristic = { 'A': 5, 'B': 4, 'C': 3, 'D': 7, 'E': 6, 'F': 2, 'G': 1 } def best_first_search(start, goal): visited = [] path = [] queue = [(start, heuristic[start])] while queue: # Sort queue based on heuristic value (lowest first) queue.sort(key=lambda x: x[1]) current, _ = queue.pop(0)</pre>

```

    if current in visited:
        continue
    visited.append(current)
    path.append(current)
    if current == goal:
        break
    for neighbor in graph[current]:
        if neighbor not in visited:
            queue.append((neighbor, heuristic[neighbor]))
    return path
# Run best first search from 'A' to 'G'
result_path = best_first_search('A', 'G')
print("Path found by Best First Search:", result_path)

```

The results (Screenshot)

```
Path found by Best First Search: ['A', 'C', 'G']
```

Lab Example No 2:

Solution:

Brief description (3-5 lines)

Beam Search Algorithm:

Beam search is a heuristic search algorithm commonly used in artificial intelligence and natural language processing, particularly in tasks such as machine translation and speech recognition. It is a variation of best-first search that aims to find the best solution by exploring a limited set of most promising paths at each step, called the "beam width".

The code

```

# Simple graph (adjacency list)
graph = {
    'A': ['B', 'C', 'D'],
    'B': ['E', 'F'],
    'C': ['G'],
    'D': ['H', 'T'],
    'E': [], 'F': [], 'G': [], 'H': [], 'T': []
}
# Heuristic values (lower is better)
heuristic = {
    'A': 5, 'B': 4, 'C': 3, 'D': 6,

```

```

'E': 7, 'F': 2, 'G': 1, 'H': 5, 'I': 4
}
def beam_search(start, goal, beam_width):
    queue = [(start, heuristic[start])]
    path = []
    while queue:
        # Sort nodes based on heuristic (ascending)
        queue.sort(key=lambda x: x[1])
        # Keep top 'beam_width' candidates
        queue = queue[:beam_width]
        next_queue = []
        for current, _ in queue:
            path.append(current)
            if current == goal:
                return path
            for neighbor in graph[current]:
                next_queue.append((neighbor, heuristic[neighbor]))
        queue = next_queue
    return path
# Run beam search with beam width 2
result = beam_search('A', 'G', beam_width=2)
print("Beam Search Path:", result)

```

The results (Screenshot)

Beam Search Path: ['A', 'C', 'B', 'G']

Lab Task No 1:

Solution:

Brief description (3-5 lines)

Perform Best First Search and beam search algorithm for finding the shortest path in a graph. Assume heuristic values on own.

'A': ['B', 'C', 'D'],

'B': ['E', 'F'],

'C': ['G', 'H'],

'D': ['I', 'J'],

'E': ['K', 'L'],

'F': [],

'G': ['M'],

```
'H': ['N'],  
'T': ['O'],  
'J': ['P']
```

The code

```
from queue import PriorityQueue  
from heapq import heappush, heappop  
# ----- Graph Definition -----  
graph = {  
    'A': ['B', 'C', 'D'],  
    'B': ['E', 'F'],  
    'C': ['G', 'H'],  
    'D': ['I', 'J'],  
    'E': ['K', 'L'],  
    'F': [],  
    'G': ['M'],  
    'H': ['N'],  
    'I': ['O'],  
    'J': ['P'],  
    'K': [],  
    'L': [],  
    'M': [],  
    'N': [],  
    'O': [],  
    'P': []  
}  
# ----- Heuristic Values (Assumed) -----  
heuristics = {  
    'A': 10, 'B': 8, 'C': 7, 'D': 6,  
    'E': 5, 'F': 4, 'G': 5, 'H': 6,  
    'I': 4, 'J': 5, 'K': 0, 'L': 1,  
    'M': 2, 'N': 3, 'O': 2, 'P': 1  
}  
# ----- Best First Search (Greedy BFS) -----  
def best_first_search(graph, heuristics, start, goal):  
    visited = set()  
    pq = PriorityQueue()  
    pq.put((heuristics[start], [start]))  
    while not pq.empty():  
        _, path = pq.get()  
        current = path[-1]
```

```

    if current in visited:
        continue
    visited.add(current)
    if current == goal:
        return path
    for neighbor in graph.get(current, []):
        if neighbor not in visited:
            new_path = path + [neighbor]
            pq.put((heuristics[neighbor], new_path))
    return None
# ----- Beam Search -----
def beam_search(graph, heuristics, start, goal, beam_width):
    queue = [(start, heuristics[start])]
    while queue:
        queue = sorted(queue, key=lambda x: x[1])[:beam_width]
        next_level = []
        for path, _ in queue:
            current = path[-1]
            if current == goal:
                return path
            for neighbor in graph.get(current, []):
                new_path = path + [neighbor]
                cost = heuristics[neighbor]
                heappush(next_level, (new_path, cost))
        queue = next_level
    return None
# ----- Example Execution -----
start_node = 'A'
goal_node = 'K'
beam_width = 2
print("Graph:", graph)
print("Heuristics:", heuristics)
print("\nStarting from '{}' to reach '{}'.format(start_node, goal_node))
# Run Best First Search
bfs_result = best_first_search(graph, heuristics, start_node, goal_node)
print("\n Best First Search Path:", bfs_result)
# Run Beam Search
beam_result = beam_search(graph, heuristics, start_node, goal_node,
beam_width)

```

```
print(" Beam Search Path (Beam Width = {}): {}".format(beam_width,
beam_result))
```

The results (Screenshot)

```
Graph: {'A': ['B', 'C', 'D'], 'B': ['E', 'F'], 'C': ['G', 'H'], 'D': ['I', 'J'], 'E': ['K', 'L'], 'F': [], 'G': ['W'], 'H': ['N'], 'I': ['O'],
'J': ['P'], 'K': [], 'L': [], 'W': [], 'N': [], 'O': [], 'P': []}
Heuristics: {'A': 10, 'B': 8, 'C': 7, 'D': 6, 'E': 5, 'F': 4, 'G': 5, 'H': 6, 'I': 4, 'J': 5, 'K': 0, 'L': 1, 'W': 2, 'N': 3, 'O': 2, 'P': 1}

Starting from 'A' to reach 'K'

Best First Search Path: ['A', 'B', 'E', 'K']
Beam Search Path (Beam Width = 2): None
```

Conclusion

In conclusion, we had done the algorithms of AI such as best first search and beam search algorithm. In this lab we implemented their function by using python and check the tree behavior of the node such as start node and goal node.